

PROGRAMMING IN OPHTHALMOLOGY · SESSION 1

Python, Data and the Principles of Machine Learning

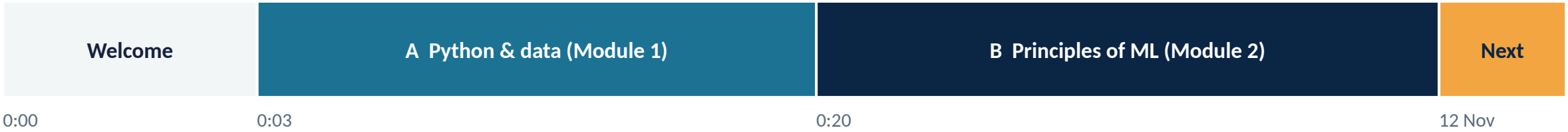
Understanding how ML works - and how to question its results

Dr. Simon König · Online Workshop, 5 November 2026, 18:00 GMT

Teaching material - not a validated clinical tool

0 WELCOME

Today: 45 minutes, two modules, one notebook



Continued in Session 2 (12 Nov): reading models critically, checklist, spot-the-flaw cases, your own data

How we will work

- Slides explain a concept - then we switch to the notebook and run it on real ophthalmic data
- No programming experience needed: you are the clinical expert, the code is a tool
- The notebook is shared after the session - rerun everything at your own pace

POLL · answer in the chat

How often do you write code?

A) never B) occasionally C) regularly

0 WELCOME

Why should clinicians understand machine learning?

ML = learning rules from examples

A model is fitted to past eyes with a known outcome and then applied to a new eye. It is statistics at scale - not magic.

You hold the domain knowledge

Which measurement is trustworthy? What does a label really mean? Does a result make clinical sense? Only you can answer.

Most failures are design failures

Wrong unit of analysis, leaky splits, circular labels - not exotic algorithms. These are checkable without being a programmer.

Python + Jupyter notebooks: code, text, results

Markdown cell: explanation, headings, questions

```
iop = 17
print("elevated?", iop > 21)
```

[▶ run with Shift+Enter]

```
Output:  elevated? False
```

Plots appear right below the code



- A notebook is a document made of cells - run top to bottom
- Free, runs in the browser (Google Colab) or locally
- Reproducible: someone else can rerun your exact analysis
- Python is the standard language for data science and ML

The toolbox we use today

pandas = tables

matplotlib / seaborn = plots

scikit-learn = ML models

Python in 90 seconds: variables, lists, functions

```
# variable: a named value
iop = 17              # mmHg
eye = "right"
is_elevated = iop > 21

# list: several values
iop_values = [14, 17, 19, 24, 16]

# function: a reusable recipe
def classify_iop(value, cutoff=21):
    if value > cutoff:
        return "elevated"
    return "normal"

for v in iop_values:
    print(v, classify_iop(v))
```

Variable

a name for a value (number, text, True/False)

List

an ordered collection - e.g. a column of measurements

Function

a recipe you write once and reuse

Library

someone else's functions: import pandas

SWITCH TO NOTEBOOK · A1

First question for every dataset: what is ONE row?

Patient	Eye	Visit	Age	RNFL µm	MD dB	Glaucoma
P1	R	1	64	71	3.2	1
P1	L	1	64	88	0.9	0
P1	R	2	65	69	3.8	1
P2	R	1	58	95	0.4	0
P2	L	1	58	93	-0.1	0
P3	L	1	71	62	9.4	1

Illustrative values (not real patients). Same colour = same patient.

- A row can be an eye, a visit, a patient, or an image - decide and write it down
- In clinical data the rows are usually NOT independent: both eyes and repeat visits belong to one person
- The unit of analysis decides how you may split, count and test (Part B)

POLL · answer in the chat

A table has 3,000 rows (eye examinations). How many independent patients does it contain?

A) 3,000
B) about 1,500
C) can't tell without the patient IDs

Rule of thumb

Always compare rows vs. unique patients vs. unique eyes - before you do anything else.

Clinical data live in different systems - merge by key

Registration
key · age · sex

OCT export
key · RNFL · MRW

Visual field export
key · MD · sLV

Diagnosis
key · glaucoma

merged table

one row per eye examination
all columns side by side

```
pd.merge(a, b,
  on=KEY,
  validate="one_to_one")
```

same row count before and after

✗ Gluing tables side by side

Exports arrive in different orders. Pasting by row position attaches the wrong visual field to the wrong eye - silently.

✓ Merge on the key

The key must identify an examination: patient + EYE + visit. Let the code verify that every key is unique.

SWITCH TO NOTEBOOK · A2 - A3

Cleaning = explicit, documented decisions

The label

What do the codes mean? Which are 'yes', which 'no', which unclear? Never recode a label you cannot explain.

Missing values

Why is it missing? Drop the row, fill it in, or mark it 'unknown' - and say which.

Impossible values

Age 250? IOP 0? Plot and describe every variable before trusting it.

Duplicates

Same patient under two IDs, repeated exports, doublets - they inflate results.

The cleaning log

step	removed	left
Keep codes 0 / 1 only	200	3,300
Drop rows w/o age	100	3,200
Drop missing features	0	3,200

Illustrative numbers (not from a real dataset) - every step is a decision a reader must be able to see.

SWITCH TO NOTEBOOK · A4

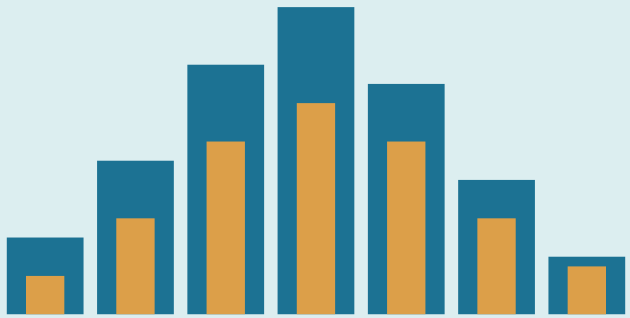
A

PART A · MODULE 1

Look before you model: three plots, three questions

Who is in my data?

Histograms, counts



age →

Do the groups differ?

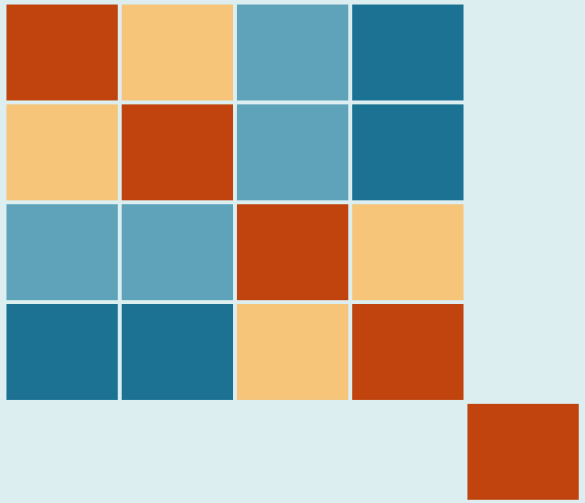
Boxplots by diagnosis



no glaucoma glaucoma

What moves together?

Correlation heatmap



features ↔ features

Schematic - the notebook draws the real ones.

SWITCH TO NOTEBOOK · A5

What is machine learning? Learning from examples

Training examples

Features X (RNFL, MD, age …)
+ known label y (glaucoma yes/no)

Learning algorithm

finds the pattern that links X to y

Model

a function: X → prediction

New eye

X known, y unknown
→ prediction (and its reliability!)

Feature (X)

an input variable

Label (y)

the outcome to predict

Training set

data the model learns from

Test set

data it has never seen

Overfitting

memorising instead of learning

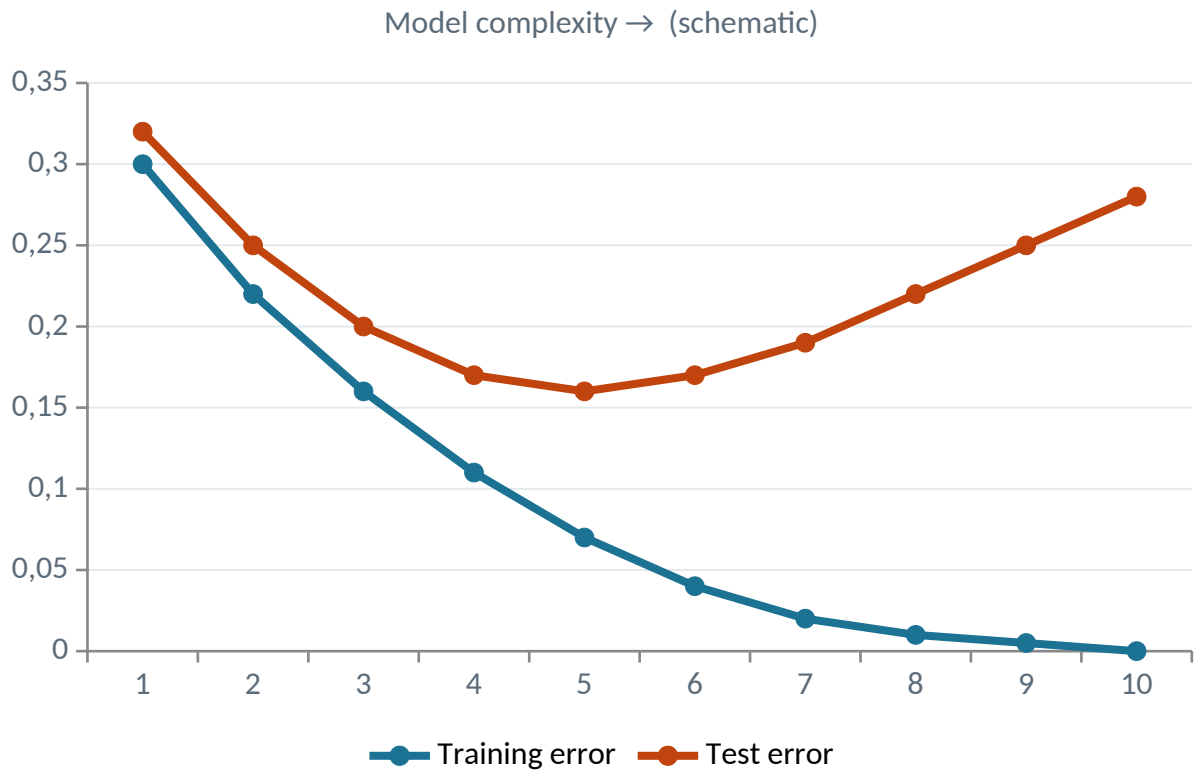
Model

the learned rule

SWITCH TO NOTEBOOK · B1

Training vs. test: don't grade the memorised exam

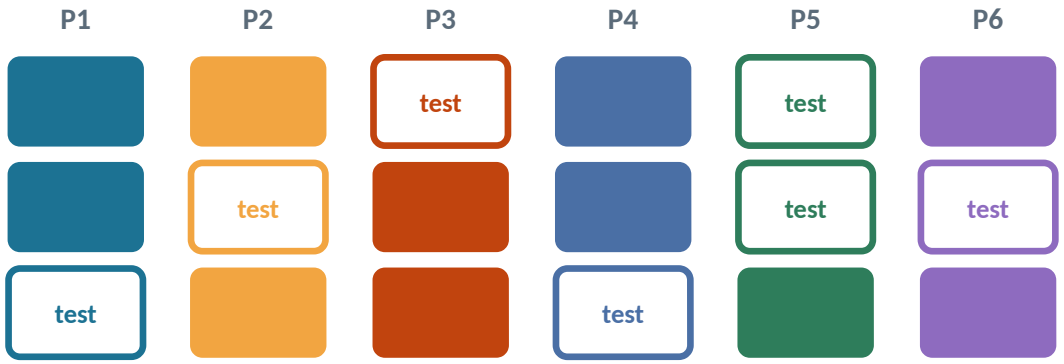
- A student who memorises last year's exam scores 100 % on it - and fails a new one
- A model can do the same: it fits noise in the training data (overfitting)
- So we keep a test set the model never sees during training
- Honest performance = performance on unseen patients



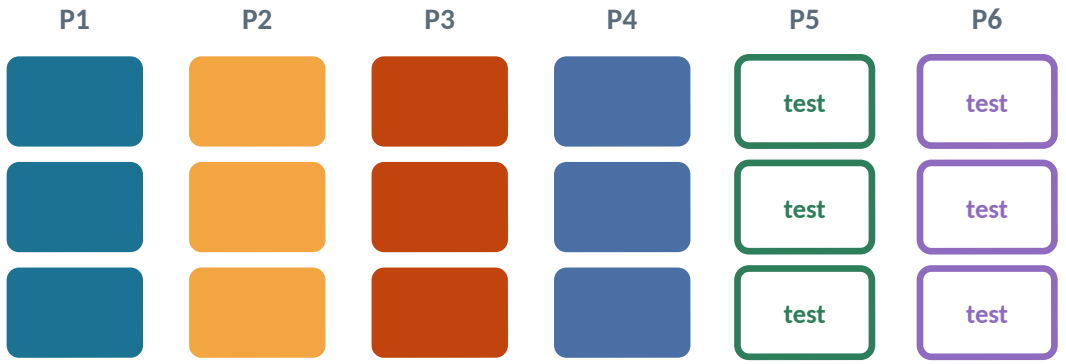
Complexity ↑ → training error keeps falling, test error rises again. The gap is overfitting.

Data leakage: the same patient on both sides of the split

✗ Random split of ROWS



✓ Split of PATIENTS



filled = training row outlined = test row each column = one patient (both eyes / visits)

Every patient appears in training AND test. The model can simply recognise the patient - the score looks great and means little.

Each patient is entirely in one set. Test = new patients, like the real future.

POLL · answer in the chat

More honest estimate?

A) random rows B) by patient

SWITCH TO NOTEBOOK · B2 - B3

Leakage has many faces - five to look for

1	**Patient / eye / image**	Several rows per person split across train and test
2	**Preprocessing before the split**	Scaling, imputation or feature selection fitted on ALL data
3	**Features that contain the answer**	Information only available after the diagnosis; features used to DEFINE the label
4	**Duplicates and doublets**	Same person under two IDs, repeated exports, near-identical images
5	**Time and place**	Train on one clinic/device/period, test on the same - not on the future or a new site

Question to ask of any ML paper: what was independent between training and test - and was it enough?

Feature scaling: put features on a common scale

Before: different units, different ranges



After standardising: mean 0, SD 1

```
z = (x − mean) / SD
```



every feature now has the same scale

Learn mean and SD from the TRAINING set only, then apply them to the test set. Otherwise test information leaks into preprocessing.

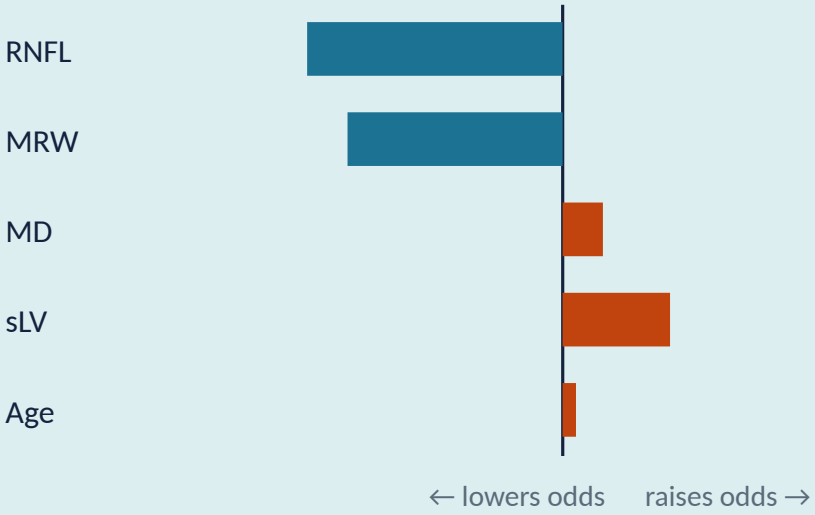
Needed for models that add up weighted features (logistic regression, neural networks). Not needed for decision trees - they only ask 'above or below a threshold?'.

SWITCH TO NOTEBOOK · B4

Two interpretable models

Logistic regression

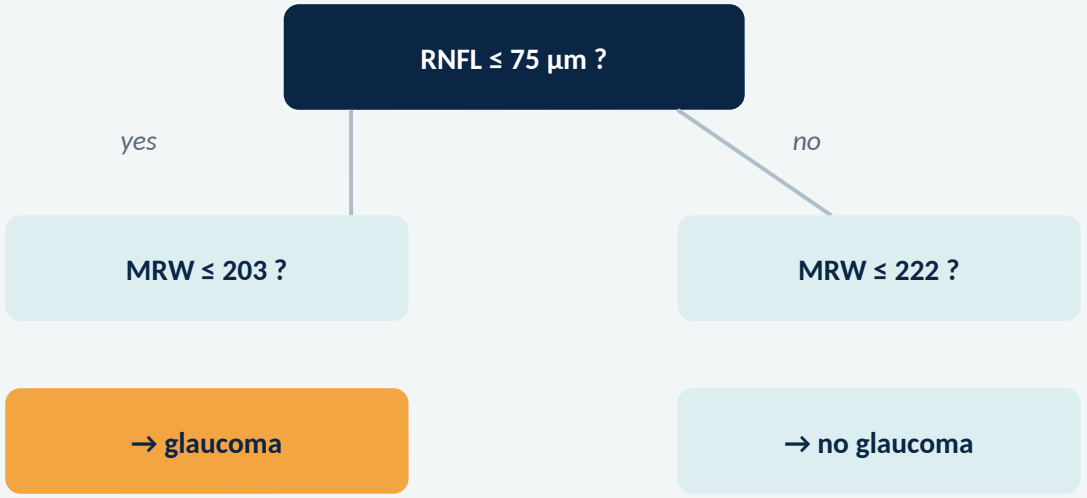
weighted sum of features → probability



Reads like an epidemiology table: direction and strength per feature (odds ratio per SD). Schematic values.

Decision tree

a flowchart of yes/no questions



Reads like a clinical flowchart. Easy to explain - but deep trees overfit.

SWITCH TO NOTEBOOK · B5